Users & Security — Authentication and RBAC

Early Access · MessageFoundry v0.3.2 · July 2026

MessageFoundry authenticates operators and authorizes every action with **role-based access control (RBAC)**, deny-by-default, on by default. It supports **local users**, **Active Directory** (LDAP bind, with optional Windows SSO), and **OIDC federation for AD-backed identities**; it maps **AD security groups to roles**, and attributes each sign-in, each ePHI access, each permission denial and each state-changing action to a named user in a tamper-evident audit trail.

RBAC is built in — not a paid add-on. Password policy ships with secure defaults, AD-group-to-role mapping is automatic, and a second authentication factor is **required by default for local accounts**.

> **Scope.** This document covers **identity, access control, and the audit of operator actions**. The protection of the *data* itself — at-rest storage and encryption, transport, logging and redaction, retention, and de-identification — is covered separately in the PHI handling guide. MessageFoundry is designed to run **inside the organization's private network**, and binds `127.0.0.1` by default; the trust boundary and the management/data/inbound plane model are described in the PHI guide and the deployment guide. Off-loopback exposure is supported, but it is a **gated posture** — see *Enforcement model* below.

Enforcement model

Authentication is **on by default, and cannot be disabled on a non-loopback bind** — that refusal is unconditional at every enforcement level, because serving full-privilege admin to the network is never one acknowledgement away. Each authenticated route additionally demands a specific **permission** — deny by default.

The off-switch, stated here rather than buried. Authentication *can* be turned off on a **loopback** bind (`[security].require_sign_in = false`) — the embedding and local-development posture. In it, every request resolves to a built-in system identity that holds every role and therefore every permission, so neither RBAC nor the field-level gate below withholds anything. It is warned at startup and named in the running posture endpoint, and a non-loopback bind with it set is a hard refusal — but it is a genuine off-switch and it belongs in the same paragraph as the claim.

A small, enumerated set of **JSON API** routes is deliberately outside that gate, and it is enumerated rather than summarized: `GET /health` (liveness), `GET /auth/providers` (which sign-in pathways this install offers, so the login page can render), `POST /auth/login` and `POST /auth/negotiate` (the sign-in ceremonies themselves, bounded by the per-IP **and** global sign-in window), and `GET /ai/policy` (so a central "AI off" can be enforced on a tokenless client). A further set of authenticated **self-service** routes — change your own password, list and

revoke your own sessions, enroll your own second factor — requires no permission because it acts only on the caller's own account. One route (`GET /service/identity`) accepts **no bearer token at all**: it authenticates by verified mTLS client certificate only.

The browser console is a **second plane with its own small unauthenticated set** — ten routes covering the sign-in, step-up and MFA pages (a gate demanding a fresh step-up in order to perform one would deadlock), plus the two OIDC legs — and a `/ui/static` **mount** carrying versioned CSS and JS, which is served with no gate at all. Every route on both planes is enumerated with its gate in the engine's security reference; nothing is left implicit.

The in-process embedding factory is **fail-closed**: with no auth service attached it denies every protected route (HTTP 503) unless the caller explicitly opts out — the deliberate embedding/local-development escape hatch. The service refuses to serve with auth disabled on a non-loopback host.

Even with auth enabled, a non-loopback bind requires **TLS** — either in-process (an API TLS certificate; TLS 1.2 floor, configurable cipher list, optional client CA for mTLS) or terminated at a trusted upstream reverse proxy (with forwarded-header and trusted-proxy configuration). The in-process option carries one further precondition, and it is a refusal rather than a warning: stdlib TLS performs no OCSP/CRL fetch and the engine deliberately attempts none (on-premises, offline by default), so a **revoked-but-unexpired** certificate would otherwise still be accepted. An off-loopback in-process TLS bind therefore refuses to start unless revocation is proven in front of it — either by declaring the revocation-checking upstream terminator, or by setting `MEFOR_TLS_REVOCATION_ATTESTED=1` to attest that your terminator or PKI enforces revocation. Loopback and proxy-terminated deployments never reach that gate. Without TLS at all the bind is refused — and on a **PHI instance at the shipped enforcement level that refusal is not one flag away**: the command-line insecure-bind escape and its configuration twin are both clamped shut there, so a staging PHI box refuses exactly as production does. Reaching a cleartext off-box bind at all requires either declaring the instance non-PHI or turning the enforcement dial down to warn — each a named switch, warned at startup and reported in the running posture endpoint. That is the accurate claim and it is the stronger one: bearer tokens and PHI cannot cross the network in cleartext by accident, and the escape that does exist is one a reviewer can read straight off the posture output. A stray host edit cannot quietly void the loopback assumption.

PHI is the default posture, and enforcement is explicit. The engine treats an instance as carrying real patient data unless you declare otherwise, and an operator-chosen **enforcement dial** decides whether a failed security precondition **refuses to start** (the default) or logs loudly and continues. Two refusals can additionally be lifted *one at a time* while the dial stays at its default — single-factor admin at exposure, and keyless PHI storage — each behind a **dedicated acknowledgement that does nothing else**, each downgrading that one refusal to a loud, audited warning, and each named in the running posture endpoint. So the refuse-or-warn decision has three inputs, not one; there is no way to reach any of them quietly.

A "synthetic" data label is a rare, warned, documented opt-out — and, since the transport-hop rules were tightened, **no data label can authorize a cleartext hop**: under enforcing configuration an undeclared plaintext egress is refused on its own terms, not excused by a classification set in a different file. The one honest escape is a per-connection acknowledgement that names the reason, and it is never silent: **every** construction of that connector emits a dedicated warning record naming the declaring connection and its stated reason, `messagefoundry check` lists the whole accepted set, and `GET /security/posture` names every connection that declares one. Be precise about that record, though — it is a distinct log line, not a row in the tamper-evident

audit table described under *Audit*, because the hop decision is configuration-level code that by design cannot reach the engine's store. (Turned down to the warning level, the enforcement dial downgrades that refusal too — as it does most others. It does not reach everything: serving with authentication off to a non-loopback bind, the ePHI-access audit floor and the in-process-TLS revocation gate above all refuse at any level. Treat that as a floor rather than the full list — the security reference is where each gate is enumerated with what does and does not lift it.)

> **Deployment guidance.** Exposing a listener off loopback always requires TLS in front of it. Plan to terminate TLS in-process or at a trusted reverse proxy before binding any interface beyond `127.0.0.1`.

First-run bootstrap admin

On first start against an empty store, the engine creates a single **bootstrap admin** (username `admin`, role `Administrator`) with a random one-time password **generated through the active password policy**. The password is **written to an owner-only file** next to the store — **never to the log** — and only the file's location is logged, so the credential is never captured in service stdout. Sign in with it, change the password immediately (the account is flagged "must change password" and enforces this), and delete the file. Once any user exists, no further bootstrap occurs.

Auto-retirement. The bootstrap account exists only to seed the first real admin, so while still **unclaimed** (never password-changed) it self-retires: it is **disabled once a second administrator exists**, and — if left unclaimed — disabled a configurable number of hours after creation (default 72 h; a value of `0` disables the timer). Once you change its password it becomes a normal admin account and is never auto-disabled, so a single-admin deployment can't lock itself out. A retired bootstrap login is refused like any other invalid credential, and the retirement is audited.

Admin password reset

An administrator (holding `users:manage`) recovers a locked-out or compromised **local** account with `POST /users/{user_id}/reset-password`. The engine generates a one-time password through the active policy, flags the account "must change password", and **revokes the user's sessions**. The temporary credential is returned **once** in the response for the admin to convey out-of-band, and the affected user is also emailed a reset notice. The administrator therefore never sets a *lasting* password the user keeps — the one-time credential must be rotated on first login.

AD users are refused (they authenticate against the directory); resetting your own account is refused (use self-service change-password). The action is audited. For the same reason, **admin-created accounts are flagged "must change password"**, so the operator's initial password is a one-time temporary that the user must rotate.

Anti-automation. Sensitive *authenticated* writes carry a **per-actor human-timing pacing floor** — a sliding window keyed on the acting user, on by default at 12 writes per second, which sits an order of magnitude above human console interaction while a machine-speed loop trips immediately. Over the floor the request is refused with `429` and a `Retry-After`, and the refusal is logged, never silent. It complements — it does not replace — the RBAC gate, step-up re-verification, the sign-in sliding window, the per-account lockout, and the per-actor PHI-read throttle.

Honest limits, because a CISO should hear it from us — including the larger one. The write-pacing floor is charged on the **JSON API only** at this release; the browser console's write routes call the JSON handlers directly and so do not charge it. The console *does* charge the per-actor PHI-read budget — but it does **not** charge the posture-keyed PHI-read *hop* refusal, so five `/ui` browse routes (the message list, a single message, its parse tree, its attachments, and the dead-letter list) can emit PHI over a serve hop the JSON API would refuse. That is the larger of the two console gaps, and naming only the smaller one would be curation rather than disclosure. Both are tracked work. All of these limiters are also **in-process**, so a multi-process deployment multiplies each budget by the number of processes — an exposed deployment must additionally front the API with a proxy or WAF limiter.

Roles & permissions

Six **fixed built-in roles** cover the common separations of duty. Each maps to permissions from the catalog below; holding multiple roles grants the **union** of their permissions (deny-by-default otherwise).

Role	Permissions
Administrator	every permission in the catalog (including `users:manage`, `audit:read`)
Operator	`monitoring:read`, `monitoring:diagnose`, `messages:read`, `messages:view_summary`, `messages:view_raw`, `messages:replay`, `messages:resend`, `messages:edit`, `messages:export`, `messages:purge`, `connections:control`, `connections:test`, `logs:view`, `files:upload`, `files:browse`, `files:delete`
Deployment	`monitoring:read`, `config:deploy`, `config:validate`, `connections:test`
Coding	`monitoring:read`, `code:edit`, `config:validate`, `ai:assist`
Viewer	`monitoring:read`, `messages:read`
Auditor	`monitoring:read`, `audit:read`, `audit:export`

Permission catalog (27): `monitoring:read`, `monitoring:diagnose`, `messages:read`, `messages:view_summary` (PHI), `messages:view_raw` (PHI), `messages:replay`, `messages:resend`, `messages:edit` (PHI), `messages:export` (PHI), `messages:purge`, `connections:control`, `connections:test`, `dr:operate`, `config:deploy`, `config:validate`, `code:edit`, `ai:assist`, `service:configure`, `users:read`, `users:manage`, `audit:read`, `audit:export`, `logs:view` (PHI), `files:upload` (PHI), `files:browse` (PHI), `files:delete`, `approvals:approve`.

Two permissions — `config:validate` and `code:edit` — have **no API endpoint yet**. They are defined so the Deployment and Coding roles are complete and those endpoints can be gated the moment they land, without a roles migration.

Note what the Operator role actually reaches. Beyond opening a single message, an Operator holds bulk raw export, edit-and-resubmit (which renders the full raw body), the redacted log tail, and the two PHI-touching uploaded-file capabilities. That is deliberate for an interface operator, and it is the role to think hardest about

when you assign it. A Viewer holds no PHI-field permission at all, so every gated property comes back `null` for them.

Custom roles

A **custom-role builder is built**, as an *additive overlay* on the six built-ins rather than a replacement. A custom role is a named **subset of the same 27-permission catalog** — it can never define a new permission kind — and it may **never** grant `users:manage`, `approvals:approve` or `dr:operate`: the escalation primitives stay admin-only. An empty set or an unknown permission string is rejected on write, and a hand-edited persisted row decodes defensively to the empty set. A caller's effective permissions are the flat union of their built-in and custom role grants. Managing custom roles requires `users:manage` plus a step-up re-verification.

> **AI coding assistance is centrally policy-governed, and `ai:assist` is its RBAC gate on the engine-brokered path.** The assistant is bounded by an environment-clamped central **policy** (`mode` from off through PHI-safe, plus `data_scope` and `environment`) read via `GET /ai/policy`. That endpoint is intentionally unauthenticated — the install policy is non-sensitive operational config that a central `off` must be able to enforce on a tokenless client, and it is honoured token or not. Be precise about the permission, though: in the shipped **bring-your-own-provider** mode the engine *reports* the caller's `ai:assist` bit as `assist_permitted` and the IDE extension enforces it, and a caller with no token (`assist_permitted: null`) is **permitted** — deliberately, because that mode sends only **code**, never message data, to the developer's own provider. `ai:assist` is enforced *in the engine* on the broker route (`POST /ai/chat`). So treat it as a client-side hint under BYO and a server-side access control on the broker path — not as one uniform gate. See the AI assistance guide.

Route → permission map (engine API)

A representative selection; the security reference enumerates every route in the product, including the browser console plane.

Endpoint(s)	Permission
`GET /health`	none (liveness)
`GET /channels`, `/connections`, `/status`, `/stats`, `/metrics`, `ws /ws/stats`	`monitoring:read`
`GET /security/posture`	`monitoring:read`
`POST /status/integrity-check`, `POST /statistics/reset`, the `/alerts` write routes	`monitoring:diagnose`
`GET /messages`	`messages:read`; `messages:view_summary` unlocks the `summary` / `error` / `metadata` fields (per-property — see *Field-level authorization*)

Endpoint(s)	Permission
`GET /messages/{id}`	`messages:view_raw` (the raw body); `messages:view_summary` unlocks `summary` / `error` / `metadata` and the nested `last_error` / event `detail` (per-property)
`GET /messages/{id}/responses`	`messages:read`; `messages:view_summary` unlocks the captured-reply `detail`, `messages:view_raw` the reply `body`
`GET /messages/export`	`messages:export` **and** `messages:view_raw` — the largest PHI egress surface, so bulk export is a capability distinct from opening one message
`POST /messages/{id}/replay`, `POST /dead-letters/replay`	`messages:replay`
`POST /messages/{id}/resend`	`messages:resend`
`POST /messages/{id}/edit-resend`	`messages:edit`
`GET /dead-letters`	`messages:read`; `messages:view_summary` unlocks the `summary` / `last_error` fields
`POST /connections/{name}/{start,stop,restart}`	`connections:control`
`GET /connections/{name}/metadata`	`monitoring:read` (per-channel for inbound; a shared outbound is barred to scoped users; credentials scrubbed unconditionally)
`POST /connections/{name}/test`	`connections:test` (reachability probe — builds a fresh connector, honors egress policy, sends no real data, audited)
`POST /connections/{name}/purge`	`messages:purge`
`POST /config/reload`	`config:deploy`
`POST /uploads`, `GET /uploads`, `DELETE /uploads/{id}`	`files:upload` / `files:browse` / `files:delete`
`GET /logs/tail`	`logs:view` (best-effort-redacted log tail; audited)
`GET /audit` / `GET /audit/export`	`audit:read` / `audit:export`
`GET` / `PUT /users/{id}/channel-scope`	`users:manage` (per-channel RBAC)
`GET` / `PUT /ad-group-map`, `GET` / `PUT /ad-group-scope-map`	`users:manage` (AD-group → role, AD-group → channel scope)
`GET /approvals`, `POST /approvals/{id}/approve`, `POST /approvals/{id}/reject`	`approvals:approve` (dual-control release/decline — see below)
`GET /service/identity`	`monitoring:read`, **mTLS client certificate only** — no bearer token is accepted, and the gate refuses at startup if it is ever asked to protect a PHI-view permission

Per-channel scoping. Operational permissions can be confined to a set of connections per user (`PUT /users/{id}/channel-scope` ; `null` = all, the default). When a user is scoped, `messages:read` / `view_raw` / `replay` , dead-letter list/replay, and `connections:control` are restricted to their channels (out-of-scope message access returns 404 to avoid leaking existence; connection control returns 403; denials are audited). **Administrators are always all-channels.** Monitoring dashboards stay global. A channel-scoped user **cannot purge** a shared outbound (purge spans every inbound feeding it). **AD users** inherit their scope from the AD-group → scope map (channel `*` = all): on login the group-derived scope is persisted and stale sessions revoked. It is opt-in — with no matching mapped group, the user's existing scope is left untouched.

`/config/reload` **executes Python** from the target directory in-process, so it is constrained beyond the `config:deploy` permission: the directory must resolve **within** an allowed root — the server's startup config directory or an entry in the allowed reload-roots list — otherwise it is rejected (403). Every reload (and every denial) is audited with the acting user; error responses are generic so a holder can't probe the filesystem via reload errors. Lock down the config/staging directories' access controls accordingly (see the service guide).

Source-network allow-list

The operator surface (JSON API, browser console, and the stats WebSocket) can be confined to a list of CIDRs or hosts. A request from outside the list is refused **403** — or a WebSocket close — in the **outermost** middleware, before routing, dependencies, the body cap and every auth check. **Loopback is always allowed**, unconditionally, so restricting the console can never lock the box out of its own console, and `GET /health` stays answerable so a locked-out operator can see the address the engine actually observes.

It is **off by default** (an empty list = no restriction). And one honest limit: behind an *undeclared* proxy or NAT, every request in the world resolves to the intermediary and this control is **inert**. The engine only detects that case and reports it on its posture endpoint; it does not close it, and this document will not describe it as if it did.

Dual-control approval for high-value actions

High-value operations can require a **second approver** before they execute — a maker-checker control. It is **opt-in and deny-by-default** (off unless enabled): a single-operator deployment is never blocked, and existing behavior is unchanged until you turn it on.

When enabled for an operation, invoking it does **not** execute inline. The request (operation, its parameters, and the requester) is **persisted**, and the endpoint returns **202** with an `approval_id` ; the action is held until a **distinct** user holding `approvals:approve` releases it via `POST /approvals/{id}/approve` . The requester can **never approve their own request** (enforced server-side, not by a client confirmation). On release the captured operation is **re-executed** and **both identities** are written to the hash-chained audit log; `POST /approvals/{id}/reject` declines it, and a request older than the configured expiry can no longer be approved. Approvers see the open queue at `GET /approvals` .

The gated set is configurable; the first cut covers the two highest-PHI-impact flows — **bulk dead-letter replay** and **connection purge**. (Console "are you sure?" prompts are client-side only and bypassable via the raw API — they are *not* a second approver and do not satisfy this control.)

Step-up re-verification on sensitive operations

A highly sensitive operation requires the caller's session to have **re-proved its credential recently** — not merely to hold a valid token. A step-up dependency refuses with **403** (header `X-Step-Up-Required: 1`) unless the session re-verified within a short window (default **300 seconds**). The **initial login counts as the first verification** (the sudo-timestamp model): the session's re-auth timestamp is stamped at login and refreshed by `POST /me/reauth`, so a session only needs to re-verify once its window lapses. `POST /me/reauth` re-checks the **local** password (argon2id) or performs a **live Active Directory re-bind** for AD accounts, so AD operators are never locked out. It is rate-limited like the password change and audited.

Gated operations (all `users:manage` admin flows plus the replay/purge/export/upload/config flows): create / delete / update user, set roles, set channel scope, admin session-revoke, admin reset-password and reset-MFA, custom-role writes, AD-group role/scope maps, dead-letter replay, message replay/resend/edit-resend, connection purge, config reload, and the uploaded-file writes **on the JSON API** — and, deliberately, the **bulk-PHI reads** (message search, bulk export, layered search, uploaded-file message listing), because they select PHI in bulk. Ordinary reads — listing users, the maps, the audit log, one message — are **not** gated.

Two exceptions worth stating rather than discovering. First, a *paced* class of mutating admin routes carries its permission and the per-actor pacing floor but **no step-up**: connection start / stop / restart, DR activate and release, approvals approve and reject, alert acknowledge and resolve, and statistics reset. Second, on the browser console — the plane operators actually use — `POST /ui/uploaded-logs/upload` and `POST /ui/uploaded-logs/file/{file_id}/resend` carry only their `files:upload` / `files:browse` permission, while their JSON twins carry the step-up, because a multipart body cannot survive the re-auth redirect. So a PHI-at-rest write and a PHI re-injection are permission-gated but not step-up-gated on that plane.

The most exploitable case is closed separately, and it ships closed: the routes that **bind or remove an authentication factor** — TOTP enroll, confirm and disable on the JSON API, passkey enroll and delete on the browser console — require a fresh proof bound to *that specific action*, single-use, rather than riding the session-wide window. So a session hijacked inside an already-open step-up window cannot enroll an attacker's authenticator. Name the switch, because it has one: that binding is `[auth].require_action_step_up`, **default on**; setting it to `false` is the documented organizational opt-out and reverts those routes to the shared 300-second window — which is precisely the hijack case above. Leave it on.

Step-up re-proves the password as a secondary verification. It **also** requires the session's second factor: an MFA-required caller is refused with `403` + `X-MFA-Required` until MFA verification succeeds, so these routes carry both a recent password re-verify **and** the MFA factor. A deliberate exception: the *enrollment* routes take a password-only step-up, because a gate an un-enrolled user cannot satisfy would stand in front of the one route that enrolls them. The step-up window composes with the dual-control approval above (the requester re-verifies; an independent approver still releases the action).

Multi-factor authentication

A second factor is built in and required for local accounts by default. The requirement ships **on**, and its default scope is **every local account** — not just administrators — including on the default loopback bind. It is enforced as an **access gate, not merely a step-up boundary**: a session that has not satisfied its second factor is refused with `403` + `X-MFA-Required` (in the browser it is confined to the MFA page) until it verifies. That covers

the authorized surface save one small, enumerated set of self-service `METHOD` /path pairs a half-authenticated session must keep in order to *escape* the gate at all — sign out, read your own account, verify your factor, rotate a must-change password, re-auth, and read your own factor status. Listing your own sessions and your own security events are deliberately **not** exempt: a pending session has proven one factor, which is exactly the attacker-holds-the-password case, and handing it the victim's session inventory would be reconnaissance. The factor-*enrollment* routes sit outside this gate too, for the reason given below — a gate an un-enrolled user cannot satisfy would stand in front of the only route that enrolls them — and they carry an action-bound password re-proof instead. An account that has already enrolled a factor must always satisfy it, whatever the scope is set to.

The documented organizational opt-outs are explicit: narrow the scope to administrators only, or turn the requirement off entirely. Both are configuration you choose, not a default you inherit.

Two factor types ship, both for **local** accounts:

- **Native RFC 6238 TOTP.** `POST /me/mfa/enroll` returns a setup key and `otpauth://` URI for an authenticator app, `POST /me/mfa/confirm` activates it and returns the **single-use recovery codes** (shown once), and `POST /auth/mfa-verify` satisfies a session's second factor with a TOTP code or a recovery code. `DELETE /me/mfa` disables it; an administrator clears a lost authenticator via `POST /users/{id}/reset-mfa` (which also revokes the user's sessions). The TOTP secret is stored **encrypted at rest** and recovery codes are **argon2id-hashed**; verification uses a constant-time compare over a **configurable skew window whose default is zero** — only the current 30-second step is accepted, the strictest replay posture. Widening it to the conventional ±1 step is an opt-in.
- **WebAuthn / FIDO2 passkeys** — the phishing-resistant factor, at the **same** boundary, through browser ceremonies on the web console (requires an optional install extra). The phishing resistance is the **origin binding**, which holds unconditionally; be precise about the *factor*, though — passkeys are asserted at `user_verification=preferred`, so for a passkey-**only** account the second factor may be device possession alone rather than possession plus a PIN or biometric. Because a non-browser client has no WebAuthn API, keep TOTP enrolled for automation and the test harness. The browser step-up stays **two-credential**: a passkey assertion satisfies the MFA leg only, and the password leg still stamps step-up freshness — a passkey never silently relaxes the password re-proof. Enrollment sits behind a **password-only** re-proof (so a stolen pre-MFA cookie can never bind an attacker's passkey); removal sits behind the full step-up, and removing the **last remaining factor while MFA is required is refused**. Passkeys mint **no recovery codes by design** — codes are phishable knowledge secrets that would undercut the phishing-resistant tier — so an administrator MFA reset is the always-available recovery path.

A required-but-unenrolled administrator is never locked out: the enrollment routes sit behind an action-bound *password* step-up, not the MFA gate, so the bootstrap admin enrolls and then satisfies it.

Enterprise/AD users get MFA through their own identity provider. A directory login is satisfied by the directory's controls (for example Entra Conditional Access or an MFA proxy) and is never prompted for an engine factor. **State this plainly, because it is the honest weakness:** on the AD and Kerberos pathways the engine issues the session MFA-satisfied and receives **no evidence** of what the directory actually enforced — so a password on the AD pathway reaches the same PHI surface as a passkey-backed local administrator. OIDC federation is the one delegated pathway carrying engine-side evidence: **by default**

(`[auth].oidc_require_mfa_claim`, shipped on) it refuses a sign-in whose *signature-verified* token asserts no configured MFA claim — an operator can switch that requirement off, but that does **not** quietly degrade the pathway to the AD posture — it fails closed: federated sessions are then minted un-verified and the MFA access gate refuses them, so federated sign-in stops working unless `[security].require_mfa` is also turned off (a breaking change in 0.3.1). Even on, that is an IdP **assertion**, cryptographically verified — not a proof that MFA was enforced, and we will not describe it as one.

When the API is bound **off loopback** with the local-account MFA requirement **explicitly turned off**, the service makes the posture explicit at startup: it refuses to start a PHI instance under enforcing configuration and warns otherwise, mirroring the keyless-store and open-egress startup gates. That refusal has exactly one lever: a **dedicated acknowledgement switch that does nothing else** (`allow_single_factor_admin_when_exposed`), which downgrades it to a loud, audited warning while enforcement stays at its default — intended for an exposure where an MFA-enforcing proxy supplies the second factor. So the remediation set is three: leave MFA required, keep the bind on loopback, or make that acknowledgement deliberately and see it named in the posture endpoint.

Administrative-interface defense-in-depth

The administrative interface is defended by **multiple independent layers**, not network-location trust alone:

1. **Source-network allow-list** (above) — refused pre-routing, before any auth check. Off by default, and inert behind an undeclared proxy.
2. **Network-location / exposed gate** — the API binds `127.0.0.1` by default, and on a PHI instance at the shipped enforcement level a non-loopback plaintext bind is refused at startup, with the insecure-bind escapes clamped shut. Lifting it means declaring the instance non-PHI or lowering the enforcement dial — both posture-reported. One layer, not the sole factor.
3. **Deny-by-default per-route RBAC** — every admin route asserts an explicit permission over an opaque bearer token; a denial is audited, and a grant on the sensitive, state-changing surface is audited too.
4. **Step-up re-verification** within a short window on the sensitive admin, replay, config and bulk-PHI routes (above) — not on every mutating route: the paced class and the two console upload/resend routes named there carry their permission without it.
5. **A genuine second authentication factor** at that boundary — TOTP or a WebAuthn passkey, so an MFA-required admin presents a real second factor, not a re-prompt of the same password.
6. **A contextual-risk signal** — when enabled, a sensitive admin action arriving from a **client IP the session has not verified from** emits an audit event and an out-of-band notice and **forces a fresh step-up**; a successful re-auth from that address re-anchors the session and clears the signal. The event and notice fire once per (session, new address), so a replayed token can't inflate the audit log. It is advisory and step-up-forcing only — it never changes an RBAC allow/deny and never blocks the non-admin request path. **Default off**, and byte-identical on a single-host loopback bind; recommended on for an off-loopback admin deployment.

Continuous identity verification underpins all of the above: every request re-resolves the user and roles from server-side state and re-checks idle/absolute timeout plus live disabled/role status, so a change made **in MessageFoundry** — a revoked role, a disabled account, a revoked session — takes effect on the next request. A

change made in the **directory** is different: it propagates via the reconciliation pass described under *Propagating a directory disable*, bounded by interval × strikes rather than immediate.

Device security-posture assessment is deployment-delegated, not built in-process: an attested/managed admin host and an **mTLS client certificate** are the posture control, consistent with the on-prem console model. The engine has no device-attestation channel and does not attempt one. This is the documented residual.

Field-level (property) authorization

Beyond gating whole *endpoints*, the API gates individual **PHI-bearing properties** within a response, so a caller can see an object without seeing its patient-identifying fields. The policy is declared in one place and enforced by a single redaction helper applied to every returned row, rather than re-implemented inline per endpoint. Be clear about what that does and does not buy: the helper is **fail-open by construction** — it must be *invoked* at each site, so a new PHI route shipped without its redaction call would leak silently. Centralization removes the risk of divergent per-endpoint rules, not the risk of a missing call. What covers the missing call is CI, described below.

Response property	Carries	Unlocked by
`summary` (message / dead-letter / detail rows)	patient identifiers (MRN / name / order)	`messages:view_summary`
`metadata` (message / detail rows)	the operator- or handler-attached user bag, an encrypted PHI-classified column	`messages:view_summary`
`error` / `last_error`, event `detail`, captured-response `detail`	exception / disposition text that can quote field values	`messages:view_summary`
`raw` (single-message body), attachments, the transformed outbound payload, captured-response `body`	the full message / reply	`messages:view_raw` (whole-body gate, at the endpoint)
bulk NDJSON export	many full bodies at once	`messages:export` **and** `messages:view_raw`

A caller lacking `messages:view_summary` receives those properties as `null`. Withholding is whole-value nulling — never partial masking. Bulk and detail reads that actually return a gated property feed a **coalesced per-actor/per-hour exposure census** in the audit trail, so a scripted bulk read can't harvest the patient census unaudited; a fully-redacted read by a caller with no PHI permission writes no census row, deliberately, so an unprivileged caller cannot amplify into unbounded audit growth.

`messages:view_raw` **is not a superset of** `messages:view_summary` — they are independent permission flags. The built-in roles *happen* to grant them nested (a role-policy convention, not a permission-model guarantee), so the **Viewer** role, holding neither, sees every gated property as `null` and is refused `GET /messages/{id}` outright. The split is nonetheless reachable — a custom role may hold `view_raw` **without** `view_summary` — which is exactly why the disposition fields sit on the `view_summary` tier: such a role still cannot reach exception text. Adding a new PHI-bearing response property means adding a row to the field map — CI asserts the map matches the documented table in **both** directions, asserts that every reachable response model is either mapped or on an

explicit reviewed no-PHI list, and exercises every redaction surface over HTTP as an unprivileged caller — that last one being the only guard that can catch a new route shipped without its redaction call, which, given the fail-open default, no map-level test can. The three guards cover the three distinct ways this gate can be forgotten.

Write side (engine → store). Exception and disposition text is also scrubbed *before* it is stored. A router or handler is user code that can raise an exception interpolating a raw message fragment, so every value written to the error/disposition columns goes through a redaction chokepoint at the runner, the connectors, **and** the store write methods — so an HL7-shaped fragment can't land in those columns. HL7-shaped content (segment dumps, multi-delimiter field runs) is cut while the exception **type** and field **name** are kept; the residual control for free-text PHI a script invents (for example a bare `"DOE^JANE"`) remains the read-side gate above plus the "never put PHI in an exception" convention. These columns are **encrypted at rest on every supported store backend** as defense-in-depth around the scrub.

Write side — not applicable by design. The API exposes **no client-writable PHI properties**: every mutation is a coarse, separately permission-gated action (`messages:replay` / `messages:resend` / `messages:edit` / `messages:purge` / `config:deploy` / `connections:control`), not a per-field write. Edit-and-resend does accept a client-supplied body, but it is submitted **whole** and re-ingressed as a new correlated message with the original left byte-identical — a coarse, step-up-gated action *on a message*, not a per-property write. So there is no per-property *write* authorization surface today. The trigger to revisit is the first endpoint that lets a client write a PHI property, at which point a writable-property-to-permission whitelist would be added alongside the read map.

Local, Active Directory, and federated sign-in

All operators share one identity model (a user's auth provider is `local` or `ad`).

- **Local users** authenticate with an argon2id-hashed password and are assigned roles explicitly (`PUT /users/{id}/roles` or the web console Users page).
- **AD users** authenticate by LDAP simple-bind over **LDAPS**. The engine binds with a service account to find the user, binds as the user to verify the password, then resolves group membership (including **nested** groups). Their roles are **re-synced from AD groups on every login** through the AD-group-to-role map, so manual role assignment does not apply to AD users. LDAPS is the default, not a structural guarantee — a plain bind and a disabled certificate check are both explicit opt-ins, and the latter is refused at startup outside a development escape.
- **Windows SSO (Kerberos)** — optional, off by default, and **experimental**. A SPNEGO exchange gives passwordless login on a domain-joined client; the resulting principal's groups are resolved the same way. It requires a server keytab/SPN and is single-leg only (no mutual authentication and no NTLM-fallback / multi-leg exchange). Every Kerberos reject path is audited. A browser-SSO session is deliberately minted **without** step-up freshness, so its first sensitive action forces an explicit credential step-up.
- **OIDC federation** — **off by default**, browser-console only, and deliberately narrow: it is a **third sign-in mechanism for an identity that already exists in on-prem AD**, not a new identity provider. A principal with no on-prem AD object is refused, and **roles come from the directory, never from a token claim**. The username is bound to an allow-listed UPN suffix; endpoints are operator-pinned with no `.well-known`

discovery; the session's absolute lifetime is capped at the verified token expiry and never extended by it. SAML, cloud-only users, a JSON/API federated path, refresh tokens and RP-initiated logout are all out of scope. One caveat worth planning for: step-up for a directory identity re-binds with a **password**, so an organization that federates *because* its users are passwordless may find those accounts cannot complete step-up.

AD-group → role mapping

An admin sets which AD groups govern which role via `GET/PUT /ad-group-map` (or the console). Group identifiers are matched case-insensitively and may be either the group **DN** or its **sAMAccountName**. A user in multiple mapped groups gets the union of those roles.

```
CN=MF-Admins,OU=Groups,DC=example,DC=com  ->  administrator
CN=MF-Ops,OU=Groups,DC=example,DC=com     ->  operator
```

Sessions

Sessions are **opaque server-side tokens** (not JWTs): the client holds the token, the store keeps only its SHA-256, so logout, expiry, and role changes take effect immediately. Each request enforces an **idle timeout** (default 30 min) and an **absolute lifetime** (default 12 h) — the pair that aligns these controls with **NIST SP 800-63B AAL2** reauthentication; raising either beyond those bounds is a documented risk deviation, not a supported hardening knob. Changing a password, disabling a user, or an AD-group/role change on re-login revokes that user's sessions. Session validation **fails closed on a backward wall-clock step** (an NTP step-back or VM snapshot revert) rather than reviving an expired token, and the idle clock is only refreshed by **user-driven** requests — a background keepalive does not keep a session alive. A configurable cap limits concurrent sessions per user (default **5**; a login beyond the cap revokes the user's oldest; `0` = unlimited).

The token is a **PHI-scoped** credential (the user's full RBAC for the session lifetime). The browser console holds it in the browser session behind a console-confined, `SameSite=Strict` cookie, with an `Origin` / `Sec-Fetch-Site` cross-site check on every state-changing request; API clients (the test harness, automation) send it as `Authorization: Bearer <token>` and keep it in memory, re-validating it against `/auth/me` before use and discarding a stale or revoked one. The API client **refuses to send credentials over plaintext HTTP to a non-loopback host** unless explicitly run in an insecure trusted-network development mode. The WebSocket prefers the header; the legacy query-parameter form is deprecated because it leaks into proxy and access logs.

Propagating a directory disable

A directory login is different from a local one: the engine mints its own opaque session and would otherwise never re-consult AD again — so disabling the account in Active Directory would **not** end the live session. A background reconciliation pass (default every **300 s**; `0` disables it, which is a declared loosening rather than a neutral choice) re-resolves directory principals still holding a session — up to a **per-pass bind budget** (200 by default, least-recently-probed first, so a large estate degrades to a longer effective interval rather than a bind

storm) — and revokes the sessions of accounts AD has disabled or deleted. Group membership is re-diffed on the same pass, so a **role demotion** takes effect without waiting for a login that may never happen. Revocations are audited.

Three safety properties, because a directory lookup returns one indistinguishable "not found" for *disabled*, *deleted*, *moved out of the search base*, and *the search base was never right*: the pass is **fail-open** (an unreachable domain controller revokes nothing, because a directory blip must not become a console outage during exactly the incident when operators need the console); it takes **two consecutive strikes** before revoking; and a **mass-revoke circuit breaker** aborts a pass whose revocation set is both large in absolute terms and broad as a proportion of the probed population, logging and auditing the abort instead. Revocation is therefore bounded by interval × strikes — extended by the probe budget on an estate with more than 200 signed-in directory users — and not immediate. That is the number for your risk register; put it there rather than claiming instant propagation.

Session inventory & targeted revocation

Users and admins can see and revoke individual sessions:

- `GET /me/sessions` — your active sessions (created / last-used / expiry / client; the current one is flagged). The session `id` is a one-way hash of the opaque token, safe to expose.
- `DELETE /me/sessions/{id}` — revoke one of **your own** sessions (ownership-checked: another user's id returns 404, never revealing or touching it).
- `DELETE /me/sessions` — "sign out everywhere else": revoke all your sessions except the current.
- `DELETE /users/{id}/sessions` (`users:manage`) — admin force-sign-out of a user (offboarding or suspected compromise).

Every targeted revoke is audited (with scope and actor). The web console surfaces this: an **Active sessions** view in the account menu lists your sessions and offers per-session revoke plus "sign out everywhere else", and the **Users** page has a **Revoke sessions** action for admin force-sign-out.

Security-event notifications

Users are notified of security-relevant changes to their account through **two** channels:

- **Out-of-band email to the affected user** (default on; it reuses the alert SMTP transport and is sent to each user's **own** address — not the operator alert distribution list). Fired on account **lockout** and the **first successful login after repeated failed attempts** (suspicious-login signals); and on **password change**, **email change**, **role change**, and **account disable** (credential changes). An email-change notice goes to the **old** address so the legitimate owner is alerted even if the change was hostile. With no SMTP configured (or for accounts with no email on file), the email is simply skipped. Emission is **best-effort** — a notification failure is logged and never blocks a login or an admin action.
- `GET /me/security-events` — a pull-based feed of the caller's own audited `auth.*` events (sign-ins, lockouts, password changes), most-recent-first, for accounts without a deliverable mailbox. It is a read-only view over the tamper-evident audit log (no new store of record). Admin-initiated changes (whose audit actor is the admin) are delivered by the email channel, not shown in this self view.

Password policy

Local passwords follow an **ASVS 5.0-aligned** policy: **minimum length 15**, **no mandatory character-class composition** (the per-class requirement flags are opt-in and default off, because ASVS forbids mandatory composition), plus **offline breached/common-password screening** (a bundled top-10k list, no live network call) and a small **context-word deny-list** (application, vendor, and HL7 terms). The policy is enforced identically on create-user and change-password and is tunable in configuration. AD passwords are governed by Active Directory.

Two further screens, both on by default and fully offline:

- **Username-in-password rejection** — a password that *contains* the user's own username (case-insensitive, for usernames of at least 4 characters) is rejected, catching common choices that a corpus can't.
- **Larger operator breach corpus** — point this at an offline list to augment the bundled top-10k: a **plaintext** file *or* an **HIBP-style SHA-1-hash export** (auto-detected), checked locally with no network call. Use a curated subset (it is loaded into memory), not the full multi-gigabyte set; a configured-but-unreadable path is warned at startup and falls back to the bundled list.

Authentication pathways — comparative strength

Pathway	Factor	Brute-force defense	Notes
Local (argon2id)	password **+** an engine-verified second factor (TOTP, recovery code, or a WebAuthn passkey), required by default for every local account; a passkey is asserted at `user_verification=preferred`, so a passkey-**only** account's second factor may be device possession alone	**per-account lockout** (5 attempts / 15 min, fed by both the password and the TOTP/recovery legs) + breach/context policy + the per-IP **and** global sign-in window	the only pathway the engine itself can lock out, and the only one with an origin-bound, phishing-resistant factor
AD (LDAPS simple-bind)	password (MFA delegated to the directory, and **unverifiable at the engine**)	the **directory's** lockout/complexity policy; engine-side, only the sign-in sliding window	password strength + lockout are the AD domain's responsibility. LDAPS is the default, not a structural guarantee: a plain bind and a disabled certificate check are each an explicit opt-in
Kerberos / SPNEGO	domain ticket (MFA delegated, unverifiable)	the **domain's** controls; passwordless on a joined client	experimental, off by default, single-leg; no engine-side password
OIDC federation (browser, AD-backed)	IdP-asserted; **by default** a sign-in whose signature-verified token asserts no configured MFA claim is refused (`oidc_require_mfa_claim`, on by default — an operator switch, and one not surfaced in the loosening register)	no engine credential to guess; the sign-in window plus the IdP's own lockout	hybrid-only; roles come from the directory, never from a token claim

Pathway	Factor	Brute-force defense	Notes
mTLS service identity (one non-interactive route)	a verified client certificate mapped through a deny-by-default allow-list; requires in-process TLS and is off unless both a client CA and an identity map are configured	not applicable — no guessable secret	no session, no MFA, no step-up, and therefore **fenced away from PHI-view permissions** by construction; it authorizes no admin operation

Lockout asymmetry: the engine's per-account lockout protects **local** accounts only. AD, Kerberos and OIDC brute-force resistance is the directory's or IdP's job — so for those deployments, set the domain lockout/complexity policy accordingly. The engine-side throttle that *does* cover them is the sliding-window sign-in limiter, per client IP **and** globally. Note that this limiter has a single switch that also disables the credential-ceremony budget: turning it off leaves the delegated pathways with **no engine-side anti-automation at all**, while local accounts keep their lockout. Treat it as a security control, not a tuning knob.

Brute-force & abuse protection

Beyond per-account lockout, the unauthenticated auth surface is **rate-limited** by an in-process sliding window — per client IP and globally — so password-spraying across many usernames (which never trips a single account's lockout) is bounded, and concurrent argon2id work is **capped** so a login flood can't exhaust the executor. A separate **per-actor credential-ceremony budget** bounds a session holder guessing at the re-proof surface, where lockout does not apply. Request bodies are capped (1 MiB outside the file-upload routes), a body with no `Content-Length` is refused, ambiguous framing is rejected, and auth request fields have length limits.

The **authenticated PHI-read endpoints** carry a **per-actor anti-automation throttle** — a sliding window keyed on the acting user, on by default at a generous cap (120 reads/min) that clears normal console and human use while bounding scripted PHI harvesting. It complements pagination and the per-access audit trail on those routes; a throttled read is logged (never silent) and returns `429`.

Two honest limits. First, **a per-IP limiter cannot stop source-IP rotation** by a directly reachable attacker; what survives rotation is the global sign-in ceiling plus the per-account lockout, and those reach the login route but not the credential re-proof surface. Second, the **ingest plane carries no message-rate limit** — inbound connections carry resource caps (connection count, frame and message size, receive timeout, source-IP allow-lists) but nothing throttles messages per second. These are all **in-process** protections; an exposed or multi-host deployment must additionally front the API with a proxy/WAF limiter and TLS.

Audit

Unconditionally, and with no switch that turns them off: **every authentication event, every permission denial, every ePHI access, and every state-changing authorization grant** is written to the durable audit log with the acting user — `auth.login_success` / `auth.login_failed` / `auth.login_locked` / `auth.logout` / `auth.permission_denied` / `auth.channel_denied`, plus `user.created` / `user.roles_changed` / `user.channel_scope_changed` / `user.deleted`, the AD-group map and scope-map updates, and the AD-scope-resync and session-revocation events. That grant record is the twin of the denial event. Read and monitoring

grants are **not** audited by default: auditing every polled console read would flood the chain, so full authorization tracing (successes included, reads included) is a single setting that ships **off** and is turned on for a deployment that wants the complete trail. PHI-view grants are excluded from that stream for a different reason — the ePHI-access path already records them, and *that* floor is unconditional. PHI access (viewing a raw message or displaying patient summaries) is recorded with the viewer. Read the trail via `GET /audit` (`audit:read`), or download a filtered CSV report with `audit:export`. **Credentials, tokens, and PHI bodies are never logged** (only ids and counts land in event detail).

Client attribution. Every row also carries the caller's network address, stamped at write time (so behind a declared trusted proxy it records the real client, not the proxy). It answers *where from*, which the trail previously could not: the actor said who, but nothing said which host pulled a bulk PHI export. An empty value means no client was in scope — an engine-internal or background write — never "unknown" and never a value inherited from another caller.

Tamper-evidence, and exactly how strong. Each audit row carries a `row_hash` chaining the previous row's hash with this row's content, so deleting, editing, or reordering any row breaks the chain. The strength of that depends entirely on whether the chain is **keyed**, so here is the precise position. On a store with an encryption key — the required configuration for a PHI instance, which refuses to start keyless at the shipped enforcement level — the chain is an **HMAC under a key derived from the store encryption key**, which never leaves the process. An attacker who can write `audit_log` rows but does not hold that key therefore cannot forge a self-consistent chain: this is forgery resistance, not merely a recomputable digest. And it is not backend-specific — the derived MAC is threaded through all three supported store backends, so a SQL Server or PostgreSQL deployment gets the same keyed chain a SQLite one does.

Without a store encryption key the chain is unkeyed, and that is the default posture rather than an exotic corner. At-rest encryption is off until a key is configured, and with no key there is nothing to derive a MAC from — so a keyless store keeps a plain SHA-256 chain. It still detects accidental or careless modification, but anyone who can write the table can recompute it: tamper-evident, not forgery-resistant. (A store that predates its key keeps its existing rows unkeyed until the explicit `messagefoundry rekey-audit` migration runs; it never re-keys silently on open.) Set a key. The client address is folded **inside** the chained payload in either form, so attribution cannot be rewritten without breaking the chain.

One limit holds in every configuration, keyed or not, and we would rather volunteer it than have a reviewer find it in `verify_audit_chain`'s own docstring: the walk proves the surviving rows chain cleanly, so deleting the *newest* rows is **not** caught by it — a truncated prefix still verifies. What catches that is comparing the chain's tip against a `(row count, head hash)` anchor recorded **out-of-band**. The walk is what `messagefoundry audit-verify` performs, so truncation detection depends on holding that anchor — or on an independent off-box copy of the rows, which is what the forwarding described below gives you.

Run the walk with `messagefoundry audit-verify` (exit 0 = intact). Rows written before the feature are chained on first start. And in every configuration this is in-database tamper-*evidence*, not prevention — restrict the store/file access controls (and run least-privilege; see the service guide) so the log can't be rewritten in the first place.

Off-box forwarding. The hash chain detects on-host tampering but lives on the same host as the data it protects; if that host is compromised, local evidence can be tampered with. The **general log** can therefore be shipped **off-box** to a syslog/SIEM collector (structured JSON supported), so an independent copy survives a host compromise. The same PHI-redaction and control-character-scrub filters apply to the forwarded stream as to stdout. The hop can be encrypted **without a local agent** — a native TLS syslog transport verifies the collector's certificate against an explicit PEM trust anchor (only that CA, not the system bundle) with hostname checking on by default, and supports mutual TLS. Forwarding turns **on** simply by naming a collector, so an operator who configures one can't silently forget an enable flag — but note the transport default is plain **UDP**, so naming a collector alone does not encrypt the hop. What protects it is a posture gate rather than the default: on an enforcing PHI instance a plaintext or unverified-TLS collector hop is **refused** unless the operator explicitly attests it. Select the TLS transport for the native encrypted hop, or point plain TCP/UDP at a local forwarding agent on loopback, which is always permitted. The **audit rows themselves** are **also** forwarded off-box: every committed audit row ships as PHI-redacted metadata to the same forwarder, across all supported store backends — so both the operational log and the tamper-evident audit trail survive a host or database compromise.

HIPAA §164.312 alignment

MessageFoundry **supports a HIPAA-compliant deployment** by providing the technical safeguards described here. Compliance is a property of the overall deployment and operating environment, not of the software alone.

- **Unique user identification** (required) — every user is a distinct account; no shared logins.
- **Person/entity authentication** (required) — local argon2id and/or AD bind; lockout on brute force; a second factor required by default for local accounts.
- **Audit controls** (required) — durable, user-attributed audit trail (append-only via the store API).
- **Automatic logoff** (addressable) — idle and absolute session timeouts.
- **Emergency access** (required) — **an organizational procedure, not a product feature, and we will not claim one.** The first-run bootstrap admin seeds the first real administrator and then retires by design — disabled once a second administrator exists, and, while still unclaimed, `[auth].bootstrap_expiry_hours` after creation (default 72; `0` turns the timer off and leaves retirement to the second-administrator condition alone) — so it is not a break-glass credential. Break-glass is a deployment responsibility: a sealed standing administrator credential, or a documented store-level recovery path.

Web console sign-in

The browser **web console** at `/ui` is the operator UI: it shows a sign-in form (Local / Active Directory, and OIDC where federation is enabled), holds the session in a console-confined `SameSite=Strict` cookie, gates every action by permission, exposes a **Users** admin page to `users:manage` holders, and offers **Sign out**. It is served same-origin, in-process by the engine and is **on by default for a local loopback bind**. Off-box it stays **opt-in**: a default-on console on an exposed instance degrades to JSON-only with a warning rather than turning a working deployment into a start failure, and an *explicitly* enabled off-box console runs a further startup ladder requiring a declared public origin (behind a proxy the Host header is client-forwardable, and the exact origin is what anchors both the cross-site check and the WebAuthn origin binding).

The former PySide6 desktop console has been **retired**; PySide6 now backs only the standalone test harness, which reaches the engine solely through the HTTP API.

Interface authentication (machine-to-machine)

Beyond operator sign-in, interface and API clients authenticate by the mechanism appropriate to the integration:

- **Mutual TLS (mTLS)** — two separate roles, worth separating because they have different reach. As a **transport gate**, pointing the API at a client CA makes a verified client certificate a precondition for every connection: that is the recommended posture for an exposed plane, and it is equally achievable at a reverse proxy. As an **identity**, a verified subject or SAN maps through a deny-by-default allow-list to a MessageFoundry principal — and *that* mapping requires **in-process** TLS (it reads the local handshake; there is no forwarded-client-certificate header), needs both a client CA file and a populated identity map, neither set by default, and today authorizes exactly **one** non-interactive service route. Note the deliberate fence: the certificate plane has no session, no MFA and no step-up, so it is refused at application startup if it is ever asked to gate a PHI-view permission. Certificate **revocation** checking is your PKI's job — strict path validation is not OCSP or CRL, so removing the allow-list entry (a config change plus restart) *is* the engine-side revocation surface, and should be treated as one.
- **OAuth 2.0 client-credentials** — for outbound REST/FHIR connectors, MessageFoundry mints a per-request bearer token via the `client_credentials` grant and re-mints on a `401`. The token endpoint is gated by egress policy.
- **SMART-on-FHIR Backend Services** — the FHIR connector authenticates to a SMART Backend Services endpoint with a signed-JWT (RS384/ES384) client assertion, opted in per connection. It is **client-only** — no App Launch flow and no authorization-server facade. Enforcing that assertion method (validating signature, audience and expiry, refusing a weaker client-secret method, replay-protecting the assertion id) is the **authorization server's** responsibility, a boundary a client engine cannot police.

MLLP-over-TLS is available per connection for HL7 v2 transport, with outbound peer verification on by default and optional client-certificate requirements inbound; the **DICOM C-STORE SCP** inbound supports DICOM-over-TLS with allowlisted calling AE titles and peer IPs, and refuses to construct a non-loopback listener with no peer control at all. A non-loopback **cleartext listener** — MLLP, DICOM, raw TCP or X12 — is refused at config load on *every* instance, not only a PHI one: the strict refusal is the shipped default. Lifting it is an instance-level decision, `serve --allow-insecure-bind`, and even that flag is **clamped** — a PHI instance under enforcing configuration refuses the cleartext listener *even with the flag set*. There is no per-connection route around a listener refusal today, and we will not offer you one: the connection-level attestation field the gate reads has no authoring surface (no transport-factory parameter, no `connections.toml` key), so it is not a lever an operator can currently pull. On the **egress** side the per-connection declaration is real and authorable — an outbound connection may declare its hop cleartext with a mandatory reason, crossing it with the warning record described above. Raw TCP and X12 have **no TLS support** at this release, so there an egress declaration is permanent and structural rather than transitional, and the listener side must be bound to loopback or firewalled and segmented at the OS level.

For outbound connectors verifying a downstream server certificate, an organization's **private internal CA** can be pinned once for the whole install (system roots only, system plus internal, or internal-only), rather than installed box-wide or repeated per connection. The anchor supplies *which roots verify the peer*; it never disables verification.

Security posture & self-assessment

MessageFoundry maintains a **documented self-assessment against OWASP ASVS 5.0 at Level 3**. The requirement inventory — 345 requirements, of which 92 are Level-3-only — has been verified against canonical ASVS 5.0.0 and is the one figure here worth quoting.

We do not publish a pass/fail count, and we will not assert coverage in words instead. The scoring is under reconciliation, and by the project's own rule no figure is quotable until the final re-score lands — an earlier edition of this page quoted one, taken from a document since marked unreliable, and it has been removed rather than restated. Saying "every control is built or carries a documented residual" would be that same figure written as prose, resting on a document a reader cannot open, so it is gone too. What a reader *can* check is the remediation ledger: findings from the re-scores are tracked as public backlog items, remediation is ongoing, and the programme has explicitly not been declared finished.

The framing that must travel with any of this: it is a **point-in-time, AI-assisted self-assessment — not a certification, not an audit, and not an independent review.** There has been **no third-party assessment, no penetration test, and no dynamic (DAST) testing** to date. We treat an independent review as a precondition we expect an adopter to require before off-loopback exposure — the internal risk record that carries this gap is available to evaluators under NDA rather than being an artifact we ask you to take on faith. To reconcile that with the exposed postures this page also documents: until such a review has run, the compensating controls we expect an off-loopback adopter to operate are the ones described above — TLS in front of every listener, the source-network allow-list, MFA left required, deny-by-default egress, and an off-box copy of the audit trail. MessageFoundry is not "certified" against any standard; NIST and OWASP issue no certificate, and no compliance outcome is guaranteed.

The full security-document set — the threat model, the ASVS assessments, the remediation plans and the risk-acceptance register — is maintained privately, on a published rule: a document is withheld only if it describes an **open, un-remediated weakness in enough detail to act on**, or names a customer or deployment. Closed findings are published, because a fixed and documented weakness is transparency rather than an attacker roadmap. Adopters, evaluators and security reviewers can request the withheld material under NDA; the rule and the contact route are in the project's security-documentation policy.

Supply-chain & CI security

Automated security scanning runs in continuous integration:

- **pip-audit** — audits the committed lockfile for known-CVE dependencies, so the audit is reproducible rather than auditing a fresh latest-resolve.
- **bandit** — Python static analysis over the engine source.
- **Dependabot** — scheduled dependency-update PRs for `pip` and GitHub Actions.

- **CodeQL** code scanning, **OpenSSF Scorecard**, and workflow static analysis, each as its own CI workflow. GitHub-native **secret scanning with push protection** is enabled at the repository level.

Every release publishes a verifiable supply chain, not a questionnaire answer: a **CycloneDX SBOM** of the engine, an **OpenVEX** document carrying our per-CVE exploitability assessments, **Sigstore signatures** over the wheel, sdist, SBOM and VEX, **SLSA build provenance** binding each artifact to its source commit, and PyPI-side PEP 740 attestations. Feed the SBOM and VEX to your own scanner and you triage real risk rather than unreachable CVEs. Verification commands are in the supply-chain guide.

A private vulnerability-disclosure policy is published with the project. A **full-history secret scan** (`gitleaks`, over the complete git history rather than just the tip) runs as a **blocking** CI gate alongside GitHub-native push protection, and the same scanner is available as a pre-commit hook so a credential is caught before it is ever committed.

Dependency lockfile

The project's dependency manifest carries lower-bound ranges; the **pinned, hashed** resolution lives in a committed lockfile and its cross-platform exported view (with per-package hashes). CI verifies the two are in sync and audits the export. For a fully reproducible, tamper-resistant install, use `pip install --require-hashes` against the exported lockfile (the SQL Server extra also needs the OS-level Microsoft ODBC Driver 18, which is not pip-installable).

Before installing the engine wheel itself, **verify its release provenance** — hash-pinning proves the bytes match the lockfile; provenance verification proves the build's origin. See the install guide.

Roadmap (not yet built)

The API endpoints that `code:edit` and `config:validate` will gate are deliberate follow-ups. Custom roles, OIDC federation, TOTP and WebAuthn MFA, API/WebSocket and MLLP TLS, mTLS service identity, per-channel RBAC and the published SBOM are all **built** — earlier editions of this page listed several of them as roadmap after they had shipped.

The structural gap, stated plainly, is the one a pre-1.0 project cannot self-supply: **independent external verification** — a third-party ASVS review, a penetration test, and DAST. See *Security posture & self-assessment* above.

MessageFoundry is an independent product and is not affiliated with or endorsed by Mirth, Corepoint, Cloverleaf, Rhapsody, or Ensemble; those names are trademarks of their respective owners. Comparative statements are offered in good faith and should be verified against each product's current documentation.